

Frontend Architecture — Next.js 16 + Zustand + SSE

Tài liệu giải thích kiến trúc client-side của hệ thống Dental AI, bao gồm tổ chức module Next.js App Router, quản lý state bằng Zustand, xử lý SSE streaming từ backend, và luồng UX xác thực hai yếu tố.

Tham chiếu mã nguồn: toàn bộ [frontend/nextjs-app/](#).

1. Công nghệ & thư viện

Thành phần	Phiên bản	Vai trò
Next.js	16.1.6	Framework React, App Router + RSC
React	19.2.4	UI library
TypeScript	5.7.3	Type safety
Tailwind CSS	v4	Styling utility-first
shadcn/ui (@radix-ui/*)	latest	Component primitives
Zustand	5.0.11	Global state manager (auth, theme)
react-markdown + remark-gfm + remark-breaks	10.x / 4.x	Render câu trả lời markdown
react-hook-form + zod	7.x / 3.x	Form validation
lucide-react	0.564.x	Icon set
sonner	1.7	Toast notifications

2. Cấu trúc thư mục

```
frontend/nextjs-app/
├── app/                                # Next.js App Router
│   ├── layout.tsx                     # Root layout: Theme provider, Toaster
│   ├── page.tsx                       # Trang chat (giao diện chính)
│   ├── login/page.tsx                 # Trang đăng nhập (+2FA step-up)
│   ├── register/page.tsx              # Trang đăng ký
│   └── globals.css                     # Tailwind + biến CSS theme
├── components/
│   ├── auth/
│   │   ├── auth-layout.tsx            # Card layout dùng chung login/register
│   │   └── password-input.tsx         # Input có nút ẩn/hiện password
│   ├── chat/
│   │   ├── chat-header.tsx            # Header: logo, theme toggle, account dialog
│   │   ├── chat-sidebar.tsx           # Danh sách session + New Chat + xóa
│   │   ├── chat-welcome.tsx           # Màn hình rỗng + gợi ý câu hỏi
│   │   ├── chat-input.tsx             # Textarea + nút gửi
│   │   └── chat-message.tsx           # Bubble render markdown + panel Sources
```

```

├── chat-loading.tsx      # Typing indicator
├── account-dialog.tsx    # Dialog thông tin tài khoản
├── account-profile-tab.tsx # Tab cập nhật họ tên
├── account-security-tab.tsx # Tab đổi mật khẩu + 2FA
├── about-dialog.tsx      # Dialog “Về ứng dụng”
├── icons/                # SVG tùy biến
├── ui/                   # shadcn generated components (button, dialog,
...)
├── theme-provider.tsx    # next-themes wrapper
├── hooks/                # Custom React hooks (business logic)
├── use-dental-chat.ts    # Hook chính: state + SSE streaming + CRUD
session
├── use-auth-redirect.ts  # Redirect nếu đã / chưa login
├── use-login.ts          # Form login + handle 2FA step-up
├── use-register.ts       # Form đăng ký
├── use-change-password.ts # Đổi mật khẩu
├── use-update-profile.ts  # Đổi họ tên
├── use-2fa.ts            # Toàn bộ flow 2FA (setup, verify, disable)
├── use-mobile.ts         # Detect viewport mobile
├── use-toast.ts          # Shadcn toast bridge
├── use-toggle.ts         # Tiện ích boolean toggle
├── lib/
├── constants.ts          # APP_CONFIG, UI_MESSAGES, SUGGESTIONS
├── validators.ts         # Validate token, input, confirm
├── stream-reader.ts       # Parser SSE chat stream
├── export-pdf.ts         # Xuất hội thoại ra PDF
├── utils.ts              # cn() merge Tailwind classes
├── stores/
├── use-auth-store.ts     # Zustand: token, user, login/2FA/register APIs
├── use-theme-store.ts    # Zustand: dark/light mode
├── styles/               # Theme vars bổ sung (nếu cần)
├── public/               # Static assets
├── .env.local            # NEXT_PUBLIC_API_URL
├── package.json
├── tsconfig.json
└── next.config.mjs

```

Nguyên tắc tổ chức

- **App Router + RSC** — Next.js 16, các route là server components mặc định; chỉ đánh dấu **"use client"** khi cần hooks hoặc event handlers.
- **Separation of concerns:** UI ở **components/**, business logic ở **hooks/**, state toàn cục ở **stores/**, tiện ích thuần ở **lib/**.
- **Không truy cập fetch trực tiếp từ UI** — mọi network call qua hook hoặc store.
- **Pattern: "dumb component + smart hook"** — **page.tsx** chỉ compose components + gọi hook **useDentalChat()**.

3. State Management — Zustand

3.1. stores/use-auth-store.ts

Store trung tâm cho xác thực, lưu `token`, `user`, và toàn bộ action auth:

Action	Endpoint	Ghi chú
<code>initialize()</code>	—	Đọc token từ <code>localStorage</code> khi app mount
<code>login(email, password)</code>	POST <code>/api/auth/login</code>	Nếu <code>requires_2fa=true</code> → trả <code>tempToken</code>
<code>verify2FALogin(tempToken, totpCode)</code>	POST <code>/api/auth/2fa/login-verify</code>	Bước 2 sau khi có TOTP code
<code>register({...})</code>	POST <code>/api/auth/register</code>	
<code>fetchProfile()</code>	GET <code>/api/auth/me</code>	Chạy sau login để cập nhật user info
<code>changePassword({...})</code>	POST <code>/api/auth/change-password</code>	
<code>updateProfile({fullName})</code>	PUT <code>/api/auth/update-profile</code>	
<code>clearToken()</code>	—	Xóa token + redirect <code>/login</code>

Token được persist vào `localStorage` với key `dental_ai_token`. Các hook/page gọi `initialize()` trong `useEffect` đầu tiên để rehydrate state sau reload.

3.2. `stores/use-theme-store.ts`

- Lưu trạng thái dark/light.
- Synchronize với `next-themes` và `localStorage`.
- Toggle qua `<ChatHeader />`.

3.3. Vì sao chọn Zustand thay vì Redux / Context?

Tiêu chí	Zustand	Redux Toolkit	React Context
Boilerplate	Rất thấp	Cao	Trung bình
Re-render control	Selector tự động	Cần <code>useSelector</code>	Re-render toàn subtree
DevTools	Có plugin	Có sẵn	Không có
Kích thước bundle	~1 KB	~12 KB	0 (built-in)

Với quy mô khóa luận (2 store nhỏ), Zustand là lựa chọn cân bằng giữa đơn giản và hiệu năng.

4. Luồng SSE Streaming — `lib/stream-reader.ts`

Khi người dùng gửi câu hỏi, frontend mở POST request tới `/api/chat` và **đọc response dưới dạng SSE stream** thay vì đợi JSON hoàn chỉnh:

```
const response = await fetch(`${API_BASE_URL}/chat`, {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    Authorization: `Bearer ${token}`,
  },
  body: JSON.stringify({ session_id, user_question, chat_history }),
});

await readChatStream(response, {
  onToken: (accumulated) => updateMessageInPlace(accumulated),
  onDone: (sources, rewrittenQuery) => attachMetadata(sources, rewrittenQuery),
});
```

Parser SSE

`readChatStream` xử lý từng chunk:

1. Dùng `response.body.getReader()` để stream raw bytes.
2. Decode UTF-8 rồi split theo `\n`.
3. Bỏ qua dòng rỗng hoặc không bắt đầu bằng `data: .`
4. `JSON.parse` payload, phân biệt 2 loại event:
 - `{ token: "..."} — nội dung chunk, append vào accumulated, gọi onToken.`
 - `{ done: true, sources: [...], rewritten_query: "..."} — kết thúc, gọi onDone.`

Tại sao append vào chuỗi `accumulated` thay vì từng token?

UI cần toàn bộ markdown hợp lệ để `react-markdown` render (ví dụ **`**bold**`** phải có cả 4 dấu `*`). Nếu render từng token riêng lẻ sẽ bị nhấp nháy và hỏng markdown.

5. Luồng UX chính

5.1. Đăng ký

```
(page) /register
└─> <RegisterForm>                ← components/ui + password-input
    └─> hook: useRegister()        ← validate password confirm (zod)
        └─> store.register() → POST /api/auth/register
            ├── success → toast + redirect /login
            └─ error   → toast error
```

5.2. Đăng nhập (có 2FA)

```

(page) /login
  ↳ <LoginForm>
    ↳ hook: useLogin()
      ↳ store.login() → POST /api/auth/login
        ↳ {access_token} → setToken + redirect /
        ↳ {requires_2fa, temp_token}
          ↳ <TOTPInput 6 số>
            ↳ store.verify2FALogin()
              ↳ POST /api/auth/2fa/login-verify
              ↳ setToken + redirect /

```

5.3. Chat

```

(page) /
  ↳ hook: useDentalChat()
    ↳ useEffect: GET /api/chat/sessions → render sidebar
    ↳ user click session → GET /api/chat/sessions/{id}/messages
    ↳ user nhập câu hỏi → POST /api/chat (SSE)
      ↳ onToken → setMessages(prev → append to last assistant msg)
      ↳ onDone → gán sources + rewrittenQuery
    ↳ user click delete session → DELETE /api/chat/sessions/{id}
    ↳ user click "Xóa tất cả" → DELETE /api/chat/sessions

```

5.4. Setup 2FA

```

Account Dialog → Tab Security → "Bật 2FA"
  ↳ useTwoFactor() → POST /api/auth/2fa/setup
    ↳ nhận {secret, qr_code (base64 SVG)}
      ↳ hiển thị QR + chuỗi secret
        ↳ user quét QR bằng Google Authenticator
          ↳ nhập mã 6 số → POST /api/auth/2fa/verify
            ↳ success → is_2fa_enabled = true

```

6. Validation — Zod + React Hook Form

lib/validators.ts

- `validateToken(token)` — kiểm tra token còn hiệu lực cơ bản (độ dài, prefix). Nếu sai → `clearToken` + `redirect`.
- `validateChatInput(text)` — chặn chuỗi rỗng, giới hạn độ dài.
- `confirmAction(message)` — wrapper `window.confirm` tránh viết lặp.

Form validation

Mỗi form (login, register, change-password) dùng `react-hook-form` + schema `zod`:

```
const schema = z.object({
  email: z.string().email("Email không hợp lệ"),
  password: z.string().min(6, "Mật khẩu phải ≥ 6 ký tự"),
});

const { register, handleSubmit, formState: { errors } } = useForm({
  resolver: zodResolver(schema),
});
```

Error messages hiển thị inline dưới input, có thêm toast cho network errors.

7. Theming & Dark Mode

- Tailwind v4 + `next-themes`.
- Biến CSS (`--background`, `--foreground`, `--primary`,...) định nghĩa trong `globals.css` — tự đổi theo `[data-theme]`.
- Toggle button trong `ChatHeader`; trạng thái persist qua `localStorage`.

8. Render câu trả lời — Markdown safe

Backend **đã cấm** LLM sinh markdown (Rule 6), nhưng vẫn có một số trường hợp frontend cần render (ví dụ gạch đầu dòng -, ngắt dòng). Lý do dùng `react-markdown`:

- Hỗ trợ GFM qua `remark-gfm` (table, strikethrough).
- Hỗ trợ `remark-breaks` → xuống dòng đơn cũng tạo `<br>` (thay vì cần `\n\n`).
- Cho phép custom component (ví dụ mở link trong tab mới).

Code tiêu biểu:

```
<ReactMarkdown remarkPlugins={[remarkGfm, remarkBreaks]}>
  {message.content}
</ReactMarkdown>
```

Panel Sources

Mỗi message của assistant có thể mở rộng xem nguồn: tiêu đề + section + URL gốc. Sources được lưu trong message và re-load khi user quay lại session.

Xuất PDF

`lib/export-pdf.ts` dựng HTML từ chat + CSS in → `window.print()` với `media="print"` stylesheet. Không dùng lib external.

9. Biến môi trường

File frontend/nextjs-app/.env.local:

```
NEXT_PUBLIC_API_URL=http://127.0.0.1:8000/api
```

- Tiền tố `NEXT_PUBLIC_` bắt buộc để biến được expose ra client-side bundle.
- Đổi giá trị → **bắt buộc restart** `npm run dev` (Next.js chỉ đọc `.env.local` lúc start).
- Trong production build, biến được inline tại thời điểm `next build`.

10. Build & Deploy

Development

```
cd frontend/nextjs-app
npm run dev                # Turbopack dev server, hot reload
```

Production

```
npm run build              # → .next/
npm run start              # Node server serve .next/
```

Khuyến nghị deploy

- **Vercel** — deploy 1-click, tự động CDN, edge caching.
- **Self-host qua Docker** — `node:20-alpine` + `npm ci --production` + `next start`.
- Nhớ cập nhật `NEXT_PUBLIC_API_URL` trỏ tới backend production (có HTTPS).

11. Hiệu năng & UX

Tối ưu	Triển khai
Streaming response	SSE → first paint sau ~0.5 s cloud / 2–3 s local
State normalization	Sessions store theo array, messages nested trong session
Optimistic UI	User message append ngay khi submit, trước khi có response
Lazy sources panel	Chỉ render khi user click "Xem nguồn"
<code>useCallback</code> / <code>useMemo</code>	Cho các handler và giá trị đắt
shadcn/ui primitives	Radix đảm bảo accessibility + animation mượt

12. Tiếng nói dành cho hội đồng

Điểm nhấn công nghệ frontend có thể trình bày:

1. **Next.js App Router + React 19** — kiến trúc hiện đại nhất 2026, sử dụng Server Components mặc định.
2. **SSE thuần** — tự parse, không dùng thư viện EventSource (vì EventSource không gửi được header Authorization).
3. **Zustand** — thay Redux để giảm boilerplate, vẫn giữ tính dự đoán được của single store.
4. **shadcn/ui + Tailwind v4** — design system có thể copy-own code, không lock vào thư viện.
5. **Graceful degradation** — mất kết nối giữa stream, UI vẫn hiển thị phần text đã tích lũy + thông báo lỗi, không trắng màn hình.